

📖 Ultimate Java & Microservices Technical Interview Revision Notes

Daily revision style: Read the idea first, then close the notes and explain it aloud in your own words. Focus on *why*, *when*, and *trade-offs*—not only definitions.

1. Collections Framework

HashMap internals

Remember: HashMap is a non-synchronized key-value data structure implementing Map. It allows one null key and multiple null values.

- Internally, it uses an array of buckets. Each bucket can contain nodes with key, value, hash, and next references.
- Default initial capacity is usually 16; the default load factor is 0.75.
- Bucket index is calculated from the key hash and table size. `hashCode()` locates a likely bucket; `equals()` confirms the exact key.
- `put()` inserts or updates a mapping. When the threshold is exceeded, the table grows—normally by doubling—and entries are redistributed.
- Before Java 8, collisions were handled with linked lists. From Java 8 onward, a heavily populated bucket may be treeified into a red-black tree when the bucket threshold is reached, provided the table is sufficiently large. This improves lookup from linked-list $O(n)$ toward tree lookup $O(\log n)$.
- Average `get()` and `put()` performance is $O(1)$. Worst-case behavior depends on collision handling.
- It is not thread-safe. Use external synchronization or `ConcurrentHashMap` when shared concurrent access is required.
- Structural modification during ordinary iteration can cause `ConcurrentModificationException`.

Interview trap: The bucket treeification threshold is 8, but treeification also depends on the table capacity; a small table may resize instead of treeifying.

ConcurrentHashMap

- Thread-safe map designed for high-concurrency access.
- Java 7 used segment-based locking. Java 8+ uses CAS operations and fine-grained synchronization around individual bins.
- Reads are highly concurrent; writes avoid locking the entire map.

- null keys and null values are not permitted because null cannot clearly represent absence during concurrent operations.
- Useful methods include `computeIfAbsent()`, `compute()`, `merge()`, and bulk traversal operations.

Collections overview

- List: ordered, duplicates allowed — `ArrayList`, `LinkedList`.
- Set: uniqueness — `HashSet`, `TreeSet`.
- Queue: processing order — `PriorityQueue`, `LinkedList`.
- Map: key-value associations — `HashMap`, `TreeMap`.
- `ArrayList`: fast random access, usually $O(1)$; middle insertion/removal is generally $O(n)$.
- `LinkedList`: node insertion can be $O(1)$ when the node is already known, but finding a position is $O(n)$.
- `TreeSet` and `TreeMap`: sorted red-black-tree structures with generally $O(\log n)$ operations.
- Prefer generics, select collections based on access patterns, and avoid unnecessary synchronization.

Fail-fast vs. weakly consistent/snapshot iteration

- **Fail-fast:** Detects many structural changes and may throw `ConcurrentModificationException`; examples include ordinary `ArrayList` and `HashMap` iterators.
- **Snapshot-style:** `CopyOnWriteArrayList` iterators read from a snapshot, so later writes are not visible to that iterator.
- **Weakly consistent:** `ConcurrentHashMap` iterators do not throw `ConcurrentModificationException` merely because concurrent updates occur; they may reflect some updates and not others.

Correction: “Fail-safe” is common interview shorthand, but it is not an official Java Collections contract. Always mention the exact iterator behavior.

2. Concurrency and Multithreading

Core ideas

- A thread is an independent execution path within a process.
- Thread safety means shared state remains correct when multiple threads access it.
- Common problems: race conditions, deadlock, starvation, and livelock.
- Java thread states: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, and `TERMINATED`.

Synchronization

- `synchronized` uses an intrinsic monitor lock associated with an object or class.
- It provides mutual exclusion and memory-visibility guarantees around the protected region.
- Keep critical sections small and use a consistent lock-ordering strategy to reduce contention and deadlock risk.

ExecutorService

`ExecutorService` manages worker threads instead of creating a new thread for every task.

- `newFixedThreadPool()`: fixed number of workers.
- `newCachedThreadPool()`: creates and reuses threads; use carefully because it can grow aggressively.
- `newScheduledThreadPool()`: delayed and periodic work.
- `submit()`: accepts `Runnable` or `Callable` and returns a `Future`.
- Always shut down executors or manage them as application lifecycle resources.

Runnable vs. Callable

Point	Runnable	Callable<V>
Method	<code>void run()</code>	<code>V call()</code>
Return value	No	Yes
Checked exceptions	Not declared by method	Can throw checked exceptions
Typical use	Fire-and-forget task	Computation with a result

CompletableFuture

- Supports asynchronous composition without deeply nested callbacks.
- Transform values with `thenApply()`.
- Consume results with `thenAccept()`.
- Chain side effects with `thenRun()`.
- Combine independent tasks with `thenCombine()` or `allOf()`.
- Handle failures with `exceptionally()`, `handle()`, or `whenComplete()`.
- Be deliberate about executors: the common pool is not always appropriate for blocking I/O.

Advanced utilities

- **Fork/Join:** Divide large CPU-bound work into smaller tasks using `RecursiveTask`, `RecursiveAction`, and `ForkJoinPool`.
- **ThreadLocal:** Gives each thread its own isolated value. Always call `remove()` in pooled-thread environments when values should not leak between requests.
- **ReentrantLock:** Explicit locking with features such as `tryLock()`, interruptible locking, and fairness options.
- **ReadWriteLock:** Multiple readers can proceed together, but writers require exclusive access.
- **Atomic classes:** CAS-based operations such as `AtomicInteger.incrementAndGet()` without ordinary locking.

3. Memory Management and JVM Tuning

Garbage collection

- Objects are collected when they are no longer reachable from GC roots.
- Generational heap layouts commonly discuss Young generation regions—Eden and Survivor—and Old generation memory.
- Minor collections usually focus on young objects; terminology for major/full collections varies by collector and JVM version.
- Important collectors include Serial, Parallel, G1, ZGC, and Shenandoah. Choose based on latency, throughput, heap size, and workload—not by habit.

JVM architecture

- **Class Loader subsystem:** loads class bytecode.
- **Runtime data areas:** heap, thread stacks, program counter registers, and class metadata areas.
- **Execution engine:** interpreter plus JIT compiler.
- **Metaspace:** native-memory area for class metadata, replacing PermGen in Java 8+.

Tuning workflow

1. Measure latency, throughput, CPU, allocation rate, and pauses.
2. Identify whether the bottleneck is CPU, memory, I/O, locking, or database access.
3. Inspect GC logs, thread dumps, heap dumps, and profiles.
4. Tune only after collecting evidence.
5. Re-test under realistic load.

Useful flags and tools: `-Xms`, `-Xmx`, `-XX:+UseG1GC`, GC logs, JFR, Java Mission Control, JVisualVM, JProfiler, JMH, and Eclipse MAT.

Shallow vs. deep copy

- **Shallow copy:** copies field values, but nested object references remain shared.
- **Deep copy:** recursively creates independent nested objects.
- Prefer immutable value objects, copy constructors, or explicit mapping over relying blindly on `clone()`.

4. Java 8+ Functional Features

Streams

- Intermediate operations such as `filter()`, `map()`, and `sorted()` are lazy.
- Terminal operations such as `collect()`, `reduce()`, and `forEach()` trigger execution.
- Streams are not data structures; they describe a computation over a source.
- Use `parallelStream()` only after measuring and confirming that the workload benefits from parallelism.

Optional

- Represents a value that may be present or absent.
- Common methods: `of()`, `ofNullable()`, `empty()`, `map()`, `flatMap()`, `filter()`, `orElse()`, `orElseGet()`, and `orElseThrow()`.
- `orElse()` evaluates its fallback eagerly; `orElseGet()` computes it lazily.
- Avoid using `Optional` for entity fields, method parameters, or serialization models unless there is a clear reason.

Lambdas and functional interfaces

- A functional interface has one abstract method. Examples: `Predicate`, `Function`, `Consumer`, and `Supplier`.
- `@FunctionalInterface` lets the compiler verify the contract.
- Method references such as `System.out::println` are concise lambda alternatives.
- Default and static interface methods allow interfaces to evolve without forcing every implementation to change.

Date and Time API

Use immutable, thread-safe JSR-310 classes such as `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant`, `Duration`, and `Period` instead of legacy `Date` and `Calendar` for new code.

5. Exception Handling

- **Checked exceptions:** must be caught or declared, for example `IOException`.
- **Unchecked exceptions:** subclasses of `RuntimeException`; examples include `NullPointerException` and `IllegalArgumentException`.
- Catch the most specific exception possible.
- Use try-with-resources for `AutoCloseable` resources.
- Preserve the original cause when translating exceptions.
- Log meaningful context once; do not swallow errors or catch `Throwable` casually.
- Avoid exposing stack traces or sensitive internal details in API responses.

Spring exception mapping

Use `@ControllerAdvice` with `@ExceptionHandler` to convert domain and validation errors into consistent JSON responses. Include fields such as timestamp, status, error code, message, path, and validation details where appropriate.

6. Design Patterns

- **Singleton:** one instance; be careful with global state and testability. `enum` is a simple safe option for many cases.
- **Factory:** centralizes object creation and hides concrete implementations.
- **Observer:** one-to-many notification; consider event-bus memory leaks and delivery guarantees.
- **Strategy:** interchangeable algorithms selected at runtime; useful for payment, pricing, validation, or routing logic.

Interview angle: Explain the problem each pattern solves, its trade-off, and where composition is better than inheritance.

7. REST APIs and Documentation

- REST emphasizes stateless interactions, resource-oriented URIs, uniform HTTP semantics, and separation of client and server concerns.
- Use GET for retrieval, POST for creation/actions, PUT for replacement, PATCH for partial update, and DELETE for removal.
- **Common statuses:** 200 OK, 201 Created, 202 Accepted, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Content, and 500 Internal Server Error.
- Design for pagination, filtering, sorting, idempotency, validation, authentication, authorization, and consistent error contracts.
- OpenAPI/Swagger can generate documentation and an interactive UI. Use annotations such as `@Operation`, `@ApiResponse`, and `@Parameter`.

8. Spring Framework Ecosystem

Core and IoC

- Spring manages beans, dependency injection, lifecycle hooks, and scopes.
- Common stereotypes: `@Component`, `@Service`, `@Repository`, and `@Controller`.
- Default scope is singleton per application context. Other scopes include prototype, request, and session.
- Prefer constructor injection for mandatory dependencies.
- Use `@PostConstruct` and `@PreDestroy` for lifecycle hooks when appropriate.
- `@Qualifier` selects one bean when multiple implementations exist; `@Primary` marks a default choice.

Spring Boot

- Provides auto-configuration, starters, embedded servers, externalized configuration, profiles, Actuator, and production conveniences.
- Use `@ConfigurationProperties` for type-safe grouped configuration.
- Use Actuator endpoints such as health and metrics carefully; restrict sensitive endpoints.
- Use profiles for environment-specific settings, not for hiding business logic.

Spring Data JPA and Hibernate

- Repositories such as `JpaRepository` simplify CRUD and derived queries.
- Use `@Transactional` around a clear unit of work.
- Prefer lazy loading for collections and watch for `LazyInitializationException`.
- Prevent N+1 queries using fetch joins, `@EntityGraph`, batch fetching, or carefully designed projections.
- Keep entities, DTOs, and API models conceptually separate when the system is complex.

Spring Security

- Configure authorization through `SecurityFilterChain`.
- Resource servers commonly validate JWTs from a trusted issuer.
- Use HTTPS, secure password hashing such as BCrypt, least privilege, and explicit authorization rules.
- Understand the difference between authentication—who the caller is—and authorization—what the caller may do.

AOP and validation

- AOP separates cross-cutting concerns such as logging, transactions, and security.
- Key terms: aspect, join point, pointcut, advice, and proxy.
- Bean Validation uses annotations such as `@NotNull`, `@Size`, `@Email`, `@Pattern`, `@Valid`, and `@Validated`.
- Handle `MethodArgumentNotValidException` consistently and return field-level validation errors.

Async support

- `@EnableAsync` and `@Async` can run methods asynchronously, but configure a bounded executor and define error handling.
- Do not assume asynchronous execution automatically improves performance; measure queueing, thread usage, and downstream capacity.

9. Microservices Architecture

- Microservices are independently deployable services with focused business boundaries.
- Communication may use REST, gRPC, Kafka, RabbitMQ, or other messaging systems.
- Benefits: independent scaling, deployment autonomy, and fault isolation.
- Costs: distributed transactions, network failures, eventual consistency, observability, versioning, and operational complexity.
- Use DDD to identify boundaries; avoid splitting services only by database tables.
- Common platform pieces: API gateway, service discovery, centralized configuration, circuit breaker, retries, timeouts, bulkheads, authentication, tracing, metrics, and centralized logs.
- Design retries carefully: retry only transient failures, use backoff and jitter, and ensure operations are idempotent.

10. Testing Frameworks

JUnit 5

Use `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`, assertions, parameterized tests, and `assertThrows()`.

Mockito

- `@Mock` creates a mock dependency.
- `@InjectMocks` creates the class under test with mocks injected.
- `when(...).thenReturn(...)` stubs behavior.
- `verify(...)` checks meaningful interactions.

- Mock boundaries, not every internal object.

Spring Boot tests

- `@SpringBootTest`: full application context; slower but broad.
- `@WebMvcTest`: controller slice.
- `@DataJpaTest`: JPA slice.
- `@RestClientTest`: HTTP client slice.
- Use `MockMvc` for controller behavior and `Testcontainers` when realistic database or broker integration is needed.

11. Useful Java Libraries

Lombok

Useful annotations include `@Getter`, `@Setter`, `@Builder`, `@NoArgsConstructor`, and `@AllArgsConstructor`. Use `@Data` carefully, especially on JPA entities, because generated equality, setters, and string methods can create unintended behavior.

Guava

Provides immutable collections, caching, multimaps, and precondition helpers. Prefer standard JDK features when they are sufficient; add dependencies for a clear benefit.

12. Apache Kafka

- Kafka is a distributed event-streaming platform.
- **Topic**: logical stream name.
- **Partition**: ordered append-only log segment and unit of parallelism.
- **Producer**: publishes records.
- **Consumer**: reads records.
- **Consumer group**: distributes partitions across consumers.
- **Broker**: Kafka server.
- Ordering is guaranteed within a partition, not across an entire topic.
- A consumer group cannot actively process more partitions in parallel than it has assigned consumers.
- Plan keys, partitions, retention, replication, consumer offsets, retries, dead-letter topics, and schema evolution.
- Use idempotent producers and design consumers to tolerate duplicate delivery.
- In Spring Kafka, common components include `KafkaTemplate` and `@KafkaListener`.

13. Java 17+ Advancements

Records

Records are concise data carriers with `final` components and generated accessors, constructor, `equals()`, `hashCode()`, and `toString()`. They are excellent for DTOs and value objects but do not make mutable component objects deeply immutable.

```
public record UserDto(Long id, String name) {}
```

Sealed classes

Sealed classes and interfaces restrict permitted subtypes and make domain hierarchies more explicit.

```
public sealed interface Payment permits CardPayment, CashPayment {}
```

Permitted subclasses must be `final`, `sealed`, or `non-sealed`.

Other modern features

- Text blocks improve readability for JSON, SQL, and multiline text.
- Pattern matching for `instanceof` reduces casting boilerplate.
- Pattern matching for `switch` enables type-aware branching.
- `var` improves local readability when the inferred type remains obvious.

14. Java 21 Features

Virtual threads

- Lightweight threads managed by the JVM, intended for high-concurrency workloads with many blocking operations.
- They make thread-per-request programming more scalable, but they do not make CPU-bound work faster.
- Avoid pinning virtual threads with long synchronized or native operations; test libraries for virtual-thread compatibility.
- Use structured, bounded resource management for databases and external services—virtual threads do not remove connection-pool limits.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    Future<String> result = executor.submit(() -> "done");  
}
```

Sequenced collections

Java 21 adds `SequencedCollection`, `SequencedSet`, and `SequencedMap`, with common encounter-order methods such as `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, and `reversed()` where supported.

Pattern matching

Pattern matching for `switch` and record patterns became final in Java 21, making type checks and record deconstruction more concise.

15. Quarkus

- Kubernetes-native Java framework optimized for fast startup, low memory usage, and cloud-native deployment.
- Works with JVM mode and native executables through GraalVM/Mandrel.
- Shifts metadata processing and much dependency-injection work to build time.
- Dev mode provides live coding and fast feedback.
- Important trade-off: native builds can require configuration for reflection, resources, proxies, and third-party libraries.
- Compare Quarkus with Spring Boot based on ecosystem, startup requirements, team expertise, deployment model, and operational constraints—not benchmarks alone.

16. GraphQL in Java

- GraphQL lets clients request exactly the fields they need through a typed schema.
- It can reduce over-fetching and under-fetching, but it does not automatically solve performance problems.
- Main schema concepts: types, queries, mutations, subscriptions, arguments, and nullability.
- The server uses resolvers/data fetchers to load values.
- Watch for the N+1 problem; use batching and data loaders.
- Add query depth/complexity limits, authorization at field or resolver boundaries, pagination, and observability.
- Java options include Spring for GraphQL and Quarkus GraphQL extensions.

17. PostgreSQL Advanced Concepts

- PostgreSQL is an open-source relational database with strong SQL support, transactions, extensibility, and sophisticated concurrency control.
- **JSONB**: stores binary JSON representation and supports indexing, commonly with GIN indexes.
- **MVCC**: readers see consistent row versions while writers create newer versions; vacuuming helps reclaim obsolete row versions.

- **Indexes:** B-tree is the common default; GIN is useful for many JSONB and array searches; GiST/SP-GiST support specialized data types and search patterns.
- **Transactions and isolation:** understand Read Committed, Repeatable Read, Serializable, locks, deadlocks, and retries.
- **EXPLAIN/EXPLAIN ANALYZE:** inspect query plans, row estimates, scans, joins, and actual execution time.
- **Connection pooling:** HikariCP reuses JDBC connections, but the pool must be sized with database capacity and workload in mind.
- Use migrations such as Flyway or Liquibase, parameterized queries, appropriate indexes, and regular monitoring.

18. Spring Cloud OpenFeign

- OpenFeign is a declarative HTTP client: define an interface and annotate it rather than writing repetitive client code.

```
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @GetMapping("/payments/{id}")
    PaymentResponse find(@PathVariable("id") Long id);
}
```

- Enable scanning with `@EnableFeignClients` where required by the application setup.
- Configure timeouts, error decoding, logging, metrics, authentication, and request correlation.
- Add resilience deliberately with circuit breakers, timeouts, retries, bulkheads, and fallbacks.
- A fallback should return a safe, meaningful result—not hide an outage silently.
- For high-throughput or highly reactive flows, compare Feign with `WebClient` and other clients.

19. gRPC and Protocol Buffers

- gRPC is a contract-first RPC framework using HTTP/2 and Protocol Buffers.
- Communication styles: unary, server streaming, client streaming, and bidirectional streaming.
- Generated stubs provide strongly typed client/server contracts across languages.
- Use deadlines, cancellation, TLS, status codes, health checks, and observability.
- Protobuf field numbers are part of the compatibility contract: do not casually reuse or change them.

20. Strings and Interning

- `String` is immutable and therefore naturally safe to share between threads.
- String literals are interned in the string pool; new `String()` creates an additional object unnecessarily in most cases.
- Use `equals()` for content comparison, not `==`.
- `StringBuilder` is generally preferred for repeated single-threaded concatenation.
- `StringBuffer` provides synchronized methods but is less commonly needed.
- Avoid repeated `+` concatenation inside large loops when a builder is clearer and more efficient.

21. DBMS: SQL and NoSQL

Relational databases

- Tables, rows, columns, primary keys, foreign keys, constraints, indexes, and transactions.
- ACID means atomicity, consistency, isolation, and durability.
- Important SQL categories: DDL, DML, DQL, and transaction control.
- Normalize to reduce duplication, then denormalize only for measured read or scalability needs.
- Use `EXPLAIN`, indexes, pagination, and transactions carefully.

NoSQL

- Document stores: MongoDB.
- Key-value stores: Redis.
- Wide-column stores: Cassandra.
- Graph databases: Neo4j.
- NoSQL systems often trade relational joins and strict schema behavior for flexible modeling and horizontal scalability.
- Design around access patterns, consistency requirements, partition keys, and failure behavior—not simply “schema-less” storage.

22. AWS Infrastructure

Lambda

Serverless, event-driven execution with automatic scaling and pay-per-use pricing. Watch cold starts, timeout limits, concurrency, idempotency, permissions, and external connection handling.

EC2

Virtual machines with control over the operating system and runtime. Combine with IAM roles, VPC security, EBS, Auto Scaling, health checks, patching, and CloudWatch.

EKS

Managed Kubernetes control plane for containerized workloads. Important areas include deployments, services, ingress/load balancing, IAM/RBAC, namespaces, secrets, autoscaling, observability, and cluster cost.

Load balancers

- **ALB:** Layer 7 HTTP/HTTPS routing.
- **NLB:** Layer 4 high-performance TCP/UDP/TLS routing.
- **GWLB:** gateway appliance integration.

Use health checks, appropriate timeouts, TLS termination, target registration, and cross-zone design.

S3

Object storage organized into buckets and objects. Common uses include backups, static assets, data lakes, and event triggers.

- Enable appropriate encryption, versioning, lifecycle rules, access logging, and least-privilege IAM.
- Use presigned URLs for controlled temporary access.
- Keep bucket policies and public access settings tightly restricted.

23. Daily Interview Revision Method

15-minute quick revision

1. Pick one section.
2. Write five keywords from memory.
3. Explain the topic in 60 seconds.
4. Solve one small coding or design example.
5. Note one trade-off and one production failure scenario.

Interview answer template

- **Definition:** What is it?
- **Internal working:** How does it work?
- **Why use it:** What problem does it solve?

- **Trade-off:** What can go wrong?
- **Project example:** Where would you use it in a Java backend or fintech system?

Final correction reminders

- Distinguish `HashMap` treeification threshold from table capacity requirements.
- Say “snapshot” or “weakly consistent” instead of treating all non-fail-fast iterators as identical.
- Do not claim virtual threads improve CPU-bound computation.
- Do not assume GraphQL removes all API performance issues; resolver design still matters.
- Do not assume PostgreSQL MVCC means vacuum is unnecessary.
- Do not enable retries or circuit breakers without timeouts, idempotency, and observability.